Department of Computer Engineering
A.Y-2025-26

Property	Details
Semester	T.E. Semester V – CMPN
Subject	Software Engineering & Web Development
Subject Professor In-charge	Prof. Divya Nimbalkar
Assisting Teachers	Prof. Divya Nimbalkar
Laboratory	M 313

VIT | Vidyalankar Institute of Technology

Group Members:

Roll No. 1	Name 1	Roll No. 2	Name 2
24102A0016	Sainath Nanaware	24102A0025	Kalpesh Sawkare
24102A0017	Dhiraj Pawar	24102A0033	Preet Panaviya

Field	Description
Experiment No.	SRS Document
Experiment Title	Preparation of System Requirement Specification

Software Requirements Specification

for

Intelligent AI-Powered E-Commerce Recommendation System

Intelligent Multi-Modal Recommendations Powered by Deep Neural Architectures

Version 1.0

Prepared by

Sainath Nanaware

Dhiraj Pawar

Kalpesh Sawkare

Preet Panaviya

Vidyalankar Institute of Technology

Mumbai Academic Year

2026–27

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
Sainath Nanaware , Dhiraj Pawar, Kalpesh Sawkare, Preet Panaviya	July 22 , 2026	Initial draft of SRS	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Intelligent AI-Powered E-Commerce Recommendation System, a deep learning-based hybrid recommendation platform. The purpose of this system is to translate the architectural findings of the survey "Deep Learning based Recommender System: A Survey and New Perspectives" (Zhang et al., 2018) into a working, IEEE 830-1998 compliant, production-oriented recommender application that combines collaborative, content-based, sequential and cold-start neural models to deliver accurate, explainable, and scalable personalized recommendations for an ecommerce/media catalogue.

This SRS is intended to be used by the development team for design and implementation, and by academic evaluators for assessing the scope, feasibility and completeness of the project.

1.2 Document Conventions

- Terminology: Technical terms (e.g., CF, NCF, CNN, RNN, AE) are defined in Appendix A: Glossary.
- Headings follow the IEEE 830-1998 structure (e.g., 1.1, 2.1).
- Requirements are labeled with unique identifiers (e.g., FR-1, UI-1, PR-1) for traceability.
- Bold text emphasizes key terms (e.g., Neural Collaborative Filtering, Attention Fusion).
- Italics denote mathematical notation, external references, or standards.
- Priority: All requirements are mandatory unless explicitly marked Medium/Low priority.

1.3 Intended Audience

- Development Team: Four students responsible for designing, training and deploying the recommendation models and web application.
- Academic Evaluators: Faculty assessing the project's scope, technical depth and adherence to the IEEE 830 format.
- Future Maintainers: Developers extending the model suite (e.g., adding new neural architectures) or the platform's API surface.

1.4 Product Scope

Intelligent AI-Powered E-Commerce Recommendation System is a web-based recommendation platform that ingests explicit ratings, implicit interactions, item metadata, review text and session click-streams to generate personalized top-N recommendations. Building on the taxonomy of deep learning-based recommender systems identified in the underlying survey, the system implements:

- A Neural Collaborative Filtering (NCF) engine for interaction-only recommendation (Section 4.2).
- A CNN-based content and review feature extractor for cold, sparse, or multimedia-rich items (Section 4.3).
- A GRU-based session recommender for anonymous/short-term intent modelling (Section 4.4).
- An autoencoder-based cold-start module for new users and items (Section 4.5).
- An attention-based hybrid fusion layer that combines all upstream signals into a single ranked list (Section 4.6).
- An administrator dashboard for monitoring model quality and managing retraining (Section 4.7).

Objectives:

- Improve recommendation accuracy over linear baselines (matrix factorization, SLIM) by exploiting non-linear representation learning.
- Mitigate the sparsity and cold-start problems using autoencoder-based reconstruction and content-based side information.
- Provide session-aware recommendations for non-authenticated visitors.
- Offer a degree of explainability via attention weights, addressing the interpretability concerns commonly raised against deep neural recommenders.

Exclusions:

- Multi-region / multi-language catalogues in the first release.
- Reinforcement-learning based real-time bidding or page-layout optimization.
- Native mobile applications (web-responsive UI only).
- Adversarial-network (GAN) based negative sampling — reserved for future work (Appendix C).

1.5 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- S. Zhang, L. Yao, A. Sun, Y. Tay, "Deep Learning based Recommender System: A Survey and New Perspectives," ACM Computing Surveys, 2018.
- X. He et al., "Neural Collaborative Filtering," WWW 2017.
- B. Hidasi et al., "Session-based Recommendations with Recurrent Neural Networks," ICLR 2016.
- S. Sedhain et al., "AutoRec: Autoencoders Meet Collaborative Filtering," WWW 2015.
- L. Zheng, V. Noroozi, P. S. Yu, "Joint Deep Modeling of Users and Items Using Reviews for Recommendation (DeepCoNN)," WSDM 2017.

2. Overall Description

2.1 Product Perspective

Intelligent AI-Powered E-Commerce Recommendation System is a new, self-contained web application developed as an academic mini-project. It is not a component of a larger commercial system, but its architecture mirrors the two broad categories of deep learning recommenders identified in the survey: (i) Recommendation with Neural Building Blocks (MLP, AE, CNN, RNN — used individually per module) and (ii) Recommendation with Deep Hybrid Models (the attention-based fusion layer in Section 4.6, which combines multiple building blocks). The system operates independently, exposing a REST API consumed by a React-based front end, and stores interaction, content and model-artifact data in a relational/object data store.

2.2 Product Functions

- User onboarding, authentication and preference-profile management.
- Real-time and batch generation of personalized top-N recommendations via Neural Collaborative Filtering.
- Content/review/image feature extraction using CNNs for cold and multimedia-rich items.

- Session-based next-item prediction using GRU networks for anonymous browsing.
- Autoencoder-based rating reconstruction to resolve cold-start scenarios.
- Attention-driven fusion of all recommendation signals into a single explainable ranked list.
- Administrator analytics dashboard for monitoring model performance and triggering retraining.

2.3 User Classes

End Users (Shoppers / Viewers)

Registered or anonymous visitors browsing the catalogue. They interact with the real-time recommendation widgets, rate/review items, and expect low-latency, relevant suggestions. Technical expertise: none required.

Administrators

Members of the development/operations team who monitor model quality (precision@K, recall@K, NDCG), manage the item catalogue, and trigger retraining pipelines. Technical expertise: moderate, familiar with the underlying ML pipeline.

Content Managers

Catalogue owners who upload new items, descriptions and images that feed the content-based feature extraction module (Section 4.3).

2.4 Operating Environment

- Platform: Web application accessible via modern browsers (Chrome, Firefox, Edge — latest two major versions) on desktop and mobile.
- Frontend: React (v18.x) with Chart.js/Recharts for analytics visualizations.
- Backend: Python (FastAPI/Flask) for the ML inference/training services; Node.js/Express for the application API gateway.
- Deep Learning Frameworks: PyTorch or TensorFlow/Keras for model training and serving, consistent with the frameworks catalogued in the underlying survey.
- Database: PostgreSQL for transactional data; a vector/embedding store (e.g., FAISS) for approximate nearest-neighbour retrieval of item embeddings.
- Deployment: Containerized services (Docker) hosted on a cloud VM or platform-as-a-service (e.g., Render, Railway) with GPU-backed training jobs run offline/batch.

2.5 Design Constraints

- Model latency: online inference (NCF + attention fusion) must complete within the bounds specified in PR-1.
- The system must use publicly available benchmark datasets (Section on Dataset, Appendix B) due to the absence of proprietary transaction data.
- Neural architectures are limited to three to four hidden layers per the plateauing effect observed for neural CF models beyond that depth.
- No cost constraint: only free-tier cloud services and open-source frameworks may be used for the academic deployment.

2.6 User Documentation

User Guide: A concise PDF explaining registration, browsing, rating and interpreting recommendation explanations.

- Developer Documentation: API reference, model-training playbooks and data-schema documentation delivered as Markdown in the project repository.
- In-app Help: Tooltips explaining "Why was this recommended?" (attention weight breakdown) and "How ratings are used."

2.7 Assumptions

- Sufficient historical interaction data (or a public benchmark dataset) is available to pretrain the collaborative and content models before go-live.
- Users will provide at least minimal implicit feedback (views/clicks) even if they do not explicitly rate items.
- The development team has adequate expertise in PyTorch/TensorFlow, React and REST API design.
- GPU compute (local or cloud, e.g., Google Colab) is available for offline model training.

3. External Interface Requirements

3.1 User Interface

- UI-1: Home Feed — displays personalized top-N recommendations with item thumbnail, title, price/rating and a short "why recommended" tag.
- UI-2: Item Detail Page — shows item content, reviews, and a "similar items" carousel powered by the content-based module.
- UI-3: Explanation Panel — visualizes the attention-weight breakdown across collaborative, content, session and cold-start signals.
- UI-4: Session Trail Widget — surfaces "continue browsing" suggestions generated by the GRU session model.
- UI-5: Admin Analytics Dashboard — displays Precision@K, Recall@K, NDCG@K, retraining schedule and catalogue health charts.
- UI-6: Responsive Design — adapts to screens from 360px (mobile) to desktop widths, WCAG 2.1 AA contrast compliant.

3.2 Hardware Interface

HI-1: None; the system operates on standard web browsers and cloud-hosted compute. Model training may leverage GPU hardware (local workstation or cloud instance) but this is not exposed as a direct hardware interface to end users.

3.3 Software Interface

ID	Interface	Purpose
SI-1	Recommendation Inference API (internal)	Exposes NCF, CNN-content, GRU-session and fusion model endpoints to the application backend (JSON over HTTPS).

SI-2	PostgreSQL Database	Stores users, items, interactions, reviews and computed recommendation logs.
ID	**Interface**	**Purpose**
SI-3	Vector Store (FAISS/Annoy)	Stores item/user embeddings for approximate nearest-neighbour retrieval.
SI-4	Model Training Pipeline (PyTorch/TensorFlow)	Offline batch jobs that retrain NCF, CNN, GRU and Autoencoder models on a schedule.
SI-5	React Frontend / REST Gateway	Delivers the responsive UI and routes requests to backend services.

3.4 Communication Interface

- CI-1: HTTPS for all client-server and inter-service communication.
- CI-2: REST/JSON for the recommendation inference API; gRPC optional for internal low-latency model-serving calls.
- CI-3: WebSocket (optional) for pushing near-real-time "trending now" updates to the home feed.

4. System Features

4.1 User Onboarding and Profile Management

4.1.1 Description and Priority

Allows a new user to register, authenticate, and build an explicit and implicit preference profile (ratings, browsing history, wish-lists) that seeds the cold-start and collaborative filtering pipelines.

Priority: High (Benefit: 8/9, Penalty: 7/9, Cost: 3/9, Risk: 2/9).

4.1.2 Stimulus/Response Sequences

- Stimulus: User registers or logs into the platform. Response: System creates/loads the user profile and initializes an embedding vector in the interaction matrix R.
- Stimulus: User rates, likes, or views an item. Response: System records the implicit/explicit feedback (rui) into the interaction store O and triggers incremental profile update.

4.1.3 Functional Requirements

FR-1: The system shall allow a user to create an account using email/password or OAuth-based signin.

FR-2: The system shall capture explicit feedback (ratings 1–5) and implicit feedback (clicks, views, dwell time, purchases).

FR-3: The system shall maintain a partially observed preference vector r(u) for every registered user.

FR-4: The system shall allow users to edit or delete stored preference data in compliance with dataprivacy requirements.

4.2 Neural Collaborative Filtering Recommendation Engine

4.2.1 Description and Priority

Generates a personalized top-N ranked list of items for a user by modelling the non-linear interaction between user and item latent factors using a Multilayer Perceptron (MLP), extending the classical matrixfactorization formulation with a Neural Collaborative Filtering (NCF) architecture.

Priority: High (Benefit: 9/9, Penalty: 9/9, Cost: 7/9, Risk: 5/9).

4.2.2 Stimulus/Response Sequences

Stimulus: User opens the home feed or a category page. Response: The NCF engine scores all candidate items rui = f(U, V embeddings | θ) and returns the top-N ranked list.

- Stimulus: A new interaction is logged. Response: The system schedules the item for inclusion in the next incremental/batch training cycle.

4.2.3 Functional Requirements

FR-5: The system shall learn dense user and item embeddings U and V from the interaction matrix R.

FR-6: The system shall pass concatenated user/item embeddings through a multi-layer feed-forward network (Neural CF layers) to model non-linear interactions.

FR-7: The system shall optimize the network using binary cross-entropy loss for implicit feedback with negative sampling, or squared loss for explicit ratings.

FR-8: The system shall generate a ranked top-N recommendation list within the performance bounds defined in PR-1.

FR-9: The system shall support fusing the neural interaction function with a generalized matrixfactorization branch (linear path) to retain memorization capability.

4.3 Content and Review-Based Feature Extraction

4.3.1 Description and Priority

Extracts latent semantic features from unstructured item content — product descriptions, user reviews, and images — using Convolutional Neural Networks (CNNs), enriching the interaction-only model with side information and mitigating sparsity.

Priority: High (Benefit: 8/9, Penalty: 6/9, Cost: 6/9, Risk: 4/9).

4.3.2 Stimulus/Response Sequences

- Stimulus: A new item (with description/reviews/image) is added to the catalogue. Response: The CNN feature-extraction pipeline generates and stores an item content embedding.
- Stimulus: A user submits a review. Response: The dual-CNN review network (user network + item network) re-computes the user and item textual representations, analogous to DeepCoNN.

4.3.3 Functional Requirements

FR-10: The system shall tokenize and embed review/description text using a pre-trained wordembedding layer prior to convolution.

FR-11: The system shall apply convolutional filters and max-pooling to extract local and global n-gram features from text content.

FR-12: The system shall extract visual features from product images using a pre-trained CNN (transfer learning) for visually-aware ranking.

FR-13: The system shall concatenate content embeddings with collaborative embeddings prior to the ranking layer described in Section 4.6.

4.4 Session-Based Sequential Recommendation

4.4.1 Description and Priority

Predicts the next item a user is likely to interact with during an active browsing session using a Gated Recurrent Unit (GRU) network, capturing short-term intent even for anonymous or first-time visitors, in the spirit of GRU4Rec.

Priority: Medium (Benefit: 7/9, Penalty: 5/9, Cost: 6/9, Risk: 5/9).

4.4.2 Stimulus/Response Sequences

- Stimulus: User clicks/views an item within the current session. Response: The session's 1-of-N encoded click sequence is fed into the GRU layer, which updates the hidden state and re-ranks candidate next items.
- Stimulus: Session ends or times out (30 minutes idle). Response: System discards the transient hidden state while persisting aggregated statistics to the long-term profile.

4.4.3 Functional Requirements

FR-14: The system shall maintain session-parallel mini-batches for real-time inference on active sessions.

FR-15: The system shall rank next-item candidates using a TOP1/BPR-style ranking loss over the GRU output layer.

FR-16: The system shall fall back to popularity-based ranking when a session has fewer than two recorded events.

FR-17: The system shall merge session-level recommendations with long-term profile recommendations for logged-in users.

4.5 Autoencoder-Based Cold-Start Handling

4.5.1 Description and Priority

Addresses the cold-start problem for new users and new items by reconstructing partially observed preference vectors through a denoising autoencoder, allowing the platform to generate meaningful recommendations before sufficient interaction history exists.

Priority: Medium (Benefit: 7/9, Penalty: 6/9, Cost: 5/9, Risk: 3/9).

4.5.2 Stimulus/Response Sequences

- Stimulus: A new item with no ratings is catalogued. Response: The autoencoder reconstructs an estimated rating vector using available side information.
- Stimulus: A user has fewer than five recorded interactions. Response: The system blends autoencoder-based estimates with popularity priors to compute the recommendation score.

4.5.3 Functional Requirements

FR-18: The system shall train an item-based and a user-based autoencoder (I-AutoRec / U-AutoRec) on partially observed vectors.

FR-19: The system shall apply masking noise to corrupt the input during training to improve robustness to missing ratings.

FR-20: The system shall incorporate side information (category, price band, region) into every autoencoder layer to further mitigate sparsity.

4.6 Attention-Based Hybrid Recommendation Fusion

4.6.1 Description and Priority

Combines the outputs of the collaborative (Section 4.2), content-based (Section 4.3), sequential (Section 4.4) and cold-start (Section 4.5) engines using a neural attention layer that learns to weight each signal's contribution per user–item pair, producing the final ranked recommendation list.

Priority: High (Benefit: 9/9, Penalty: 8/9, Cost: 7/9, Risk: 6/9).

4.6.2 Stimulus/Response Sequences

Stimulus: All upstream engines return candidate scores for a user. Response: The attention layer computes normalized weights over each candidate signal and produces a single fused relevance score.

- Stimulus: Fused ranking is generated. Response: The API layer serves the final top-N list along with an attention-weight breakdown for explainability.

4.6.3 Functional Requirements

FR-21: The system shall compute item-level and component-level attention weights over the available recommendation signals.

FR-22: The system shall re-rank the fused candidate list using the weighted sum of individual engine scores.

FR-23: The system shall expose the top contributing signal(s) for each recommendation to support explainable-AI requirements (see OR-4).

4.7 Administrator Analytics and Model Management Dashboard

4.7.1 Description and Priority

Provides administrators with visibility into recommendation quality, model retraining status, and catalogue health, and allows manual override of business rules (e.g., promotional boosting).

Priority: Medium (Benefit: 6/9, Penalty: 5/9, Cost: 4/9, Risk: 2/9).

4.7.2 Stimulus/Response Sequences

- Stimulus: Administrator logs into the dashboard. Response: System displays model performance metrics (precision@k, recall@k, NDCG) and retraining schedule.
- Stimulus: Administrator triggers a manual retraining job. Response: System queues an offline training job for the selected model and reports completion status.

4.7.3 Functional Requirements

FR-24: The system shall display offline evaluation metrics (Precision@K, Recall@K, NDCG@K, Hit Ratio) for each deployed model.

FR-25: The system shall allow administrators to schedule or manually trigger model retraining pipelines.

FR-26: The system shall log all administrator actions for auditability.

5. Non-Functional Requirements

5.1 Performance

PR-1: The attention-fused recommendation list shall be generated and returned within 300 ms (p95) for the online NCF + fusion path.

PR-2: Offline retraining of the NCF and CNN models shall complete within 4 hours on a single GPU for a catalogue of up to 1 million interactions.

PR-3: The platform shall support at least 500 concurrent recommendation requests with less than 1 second p99 latency.

PR-4: Session-based (GRU) inference shall update the next-item ranking within 150 ms of a new click event.

5.2 Safety

SAF-1: The system shall not surface recommendations for age-restricted or flagged items to users who have not confirmed eligibility.

SAF-2: Model outputs shall be bounded/clipped to prevent numerically unstable scores from propagating to the ranking layer.

5.3 Security

SEC-1: All API communication shall use HTTPS/TLS 1.2 or higher.

SEC-2: User passwords shall be stored using a salted one-way hash (e.g., bcrypt).

SEC-3: Access to the administrator dashboard and model-retraining endpoints shall require role-based authentication.

SEC-4: Personally identifiable information used for model training shall be anonymized/pseudonymized before being persisted to training datasets.

5.4 Quality Attributes

QA-1: Usability: New users shall be able to obtain a first recommendation without any onboarding tutorial.

QA-2: Reliability: The recommendation service shall maintain 95% uptime during evaluation/demo periods.

QA-3: Maintainability: Each neural module (NCF, CNN, GRU, AE, Attention) shall be independently deployable and testable, per the modular "neural building block" philosophy of the underlying survey.

QA-4: Explainability: Every top-N recommendation shall be traceable to at least one contributing signal via the attention-weight breakdown.

QA-5: Scalability: The architecture shall support incremental addition of new neural building blocks (e.g., a future GAN-based sampler) without redesigning the fusion layer.

5.5 Business Rules

BR-1: Promoted/sponsored items may only be boosted by a bounded multiplier over the modelcomputed relevance score, never replacing it outright.

BR-2: Recommendations shall never be based solely on a single data point (e.g., one click) without corroborating signal, to avoid overfitting to noise.

BR-3: All datasets used for training shall be either institutionally owned or public benchmark datasets with permissive licenses.

6. Other Requirements

OR-1: The project shall comply with academic submission guidelines, including source code, this SRS document, and a live demonstration.

OR-2: Source code shall be hosted on a public Git repository (e.g., GitHub) for evaluation.

OR-3: The system shall log model version, hyperparameters and training dataset snapshot for reproducibility.

OR-4: The system shall provide a human-readable explanation string alongside every recommendation (see QA-4 and FR-23).

Appendix A: Glossary

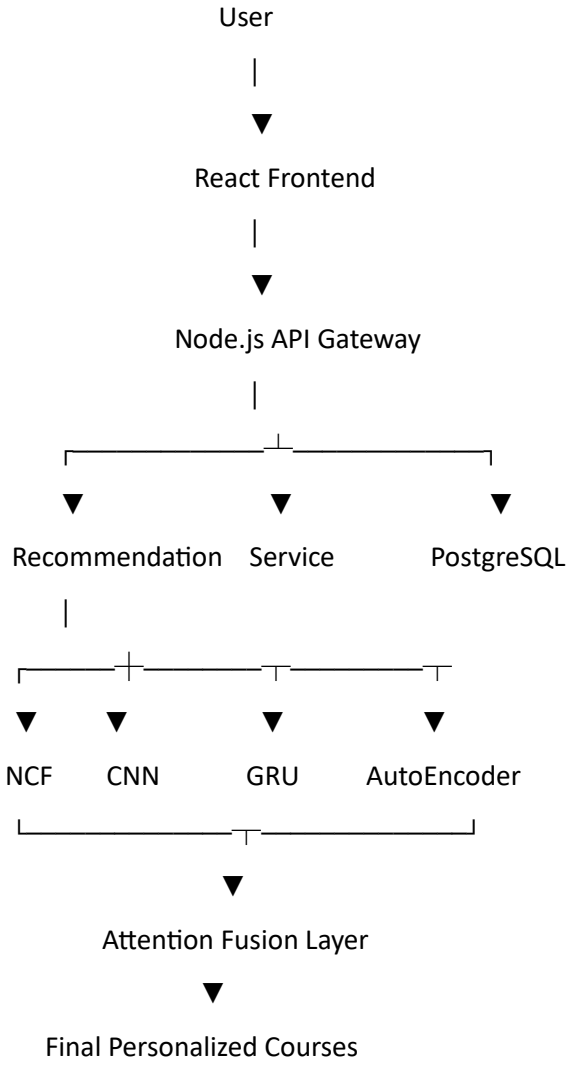
Term	Definition
CF	Collaborative Filtering — recommending based on historical user-item interaction patterns.
DL	Deep Learning — a sub-field of machine learning based on multi-layer neural representation learning.
MLP	Multilayer Perceptron — a feed-forward neural network with one or more hidden layers.
NCF	Neural Collaborative Filtering — a neural network that models the user-item interaction function.
CNN	Convolutional Neural Network — used here to extract features from review text and product images.
RNN / GRU	Recurrent Neural Network / Gated Recurrent Unit — used for sequential/session modelling.
AE	Autoencoder — an unsupervised network used to reconstruct partially observed rating vectors for cold-start handling.
Attention	A mechanism that learns to weight the contribution of different input signals or embeddings.
Embedding	A dense, low-dimensional vector representation of a user or item.
Cold-Start	The problem of recommending to a new user or item with little or no interaction history.
Top-N	The ranked list of the N most relevant items returned to a user.
NDCG	Normalized Discounted Cumulative Gain — a ranking-quality evaluation metric.

Appendix B: System Architecture and Analysis Models

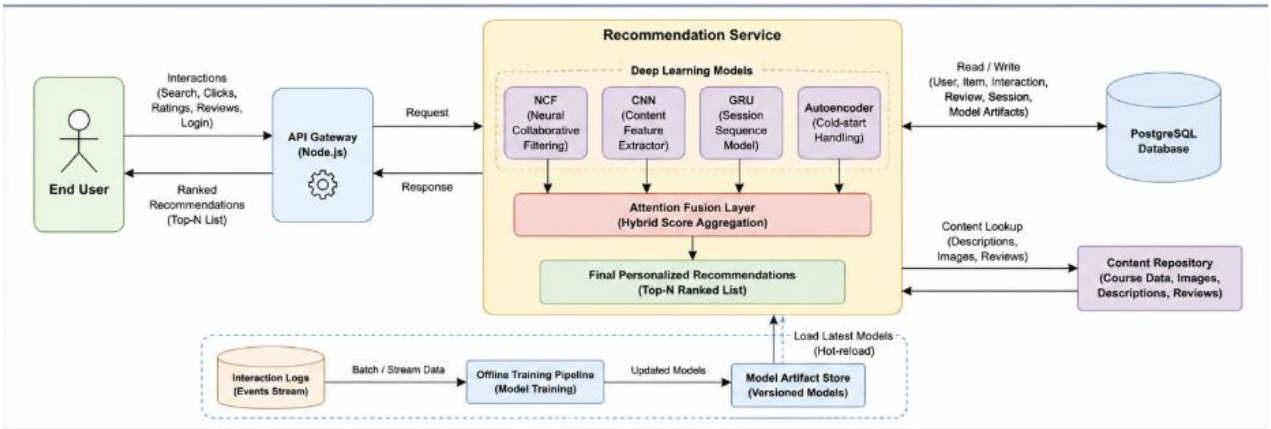
B.1 System Architecture

Layered Architecture
Presentation Layer — React web application (home feed, item detail, explanation panel, admin dashboard).
API Gateway Layer — Node.js/Express REST gateway handling authentication, routing and rate limiting.

Recommendation Service Layer — FastAPI microservices exposing NCF, CNN-content, GRU-session, Autoencoder and Attention-Fusion endpoints.
Data Layer — PostgreSQL (users, items, interactions, reviews) and a vector store (FAISS) for embeddings.
Offline Training Layer — Scheduled PyTorch/TensorFlow batch jobs that retrain each neural building block and publish new model artifacts.



B.2 Data Flow Diagram (Level 0)



Data Flow
User Interaction → API Gateway → Recommendation Service (feature lookup + model inference) → Ranked List → API Gateway → User Interface.
Item/Review/Image Upload → Content Feature Extraction (CNN) → Item Embedding Store.
Interaction Logs → Offline Training Pipeline → Updated Model Artifacts → Recommendation Service (hot-reload).
Session Click Stream → GRU Session Service → Session-Level Ranking → Fusion Layer.

B.3 Use Case Diagram

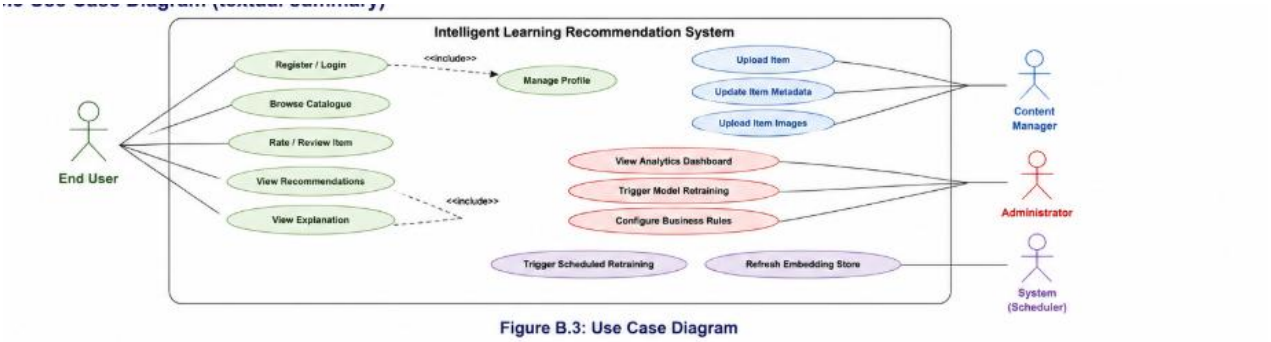
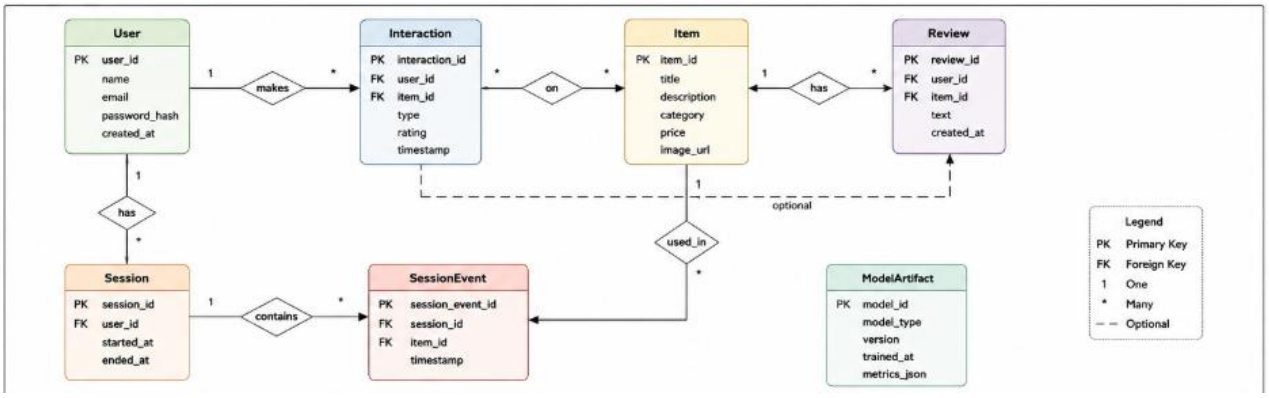


Figure B.3: Use Case Diagram

Actors and Use Cases
End User: Register/Login, Browse Catalogue, Rate/Review Item, View Recommendations, View Explanation.
Content Manager: Upload Item, Update Item Metadata, Upload Item Images.
Administrator: View Analytics Dashboard, Trigger Model Retraining, Configure Business Rules.
System (Scheduler): Trigger Scheduled Retraining, Refresh Embedding Store.

B.4 ER Diagram



Entities and Relationships
User (user_id, name, email, password_hash, created_at) 1—* Interaction.
Item (item_id, title, description, category, price, image_url) 1—* Interaction.
Interaction (interaction_id, user_id FK, item_id FK, type, rating, timestamp).
Review (review_id, user_id FK, item_id FK, text, created_at) 1—1 Interaction (optional).
Session (session_id, user_id FK nullable, started_at, ended_at) 1—* SessionEvent (item_id FK, timestamp).
ModelArtifact (model_id, model_type, version, trained_at, metrics_json).

B.5 Sequence Diagram (textual summary — Recommendation Request)

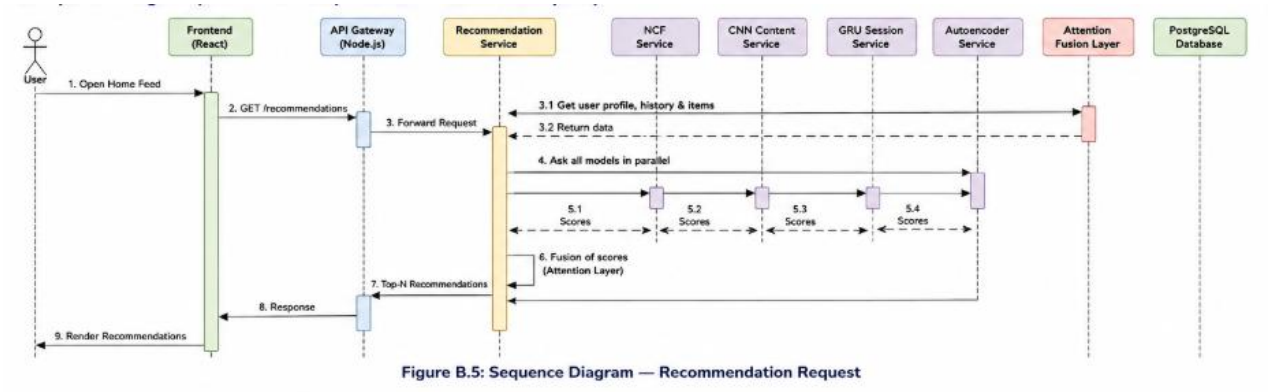


Figure B.5: Sequence Diagram — Recommendation Request

Sequence: Home Feed Recommendation
1. User opens home feed → Frontend sends GET /recommendations to API Gateway.
2. API Gateway authenticates request and forwards to Recommendation Service.

3. Recommendation Service queries embedding store for user/item vectors.
4. NCF, CNN-content and GRU-session sub-services return candidate scores in parallel.
5. Attention-Fusion layer combines scores and produces final ranked list with explanations.
6. Recommendation Service returns JSON payload to API Gateway.
7. API Gateway returns response to Frontend, which renders the home feed.

Appendix C: To Be Determined List and Future Enhancements

C.1 To Be Determined

- TBD-1: Final choice of benchmark dataset (MovieLens-1M vs. Amazon Product Reviews) pending data-availability confirmation.
- TBD-2: Choice of deep learning framework (PyTorch vs. TensorFlow/Keras) pending team familiarity assessment.
- TBD-3: Exact embedding dimensionality (k) for user/item latent factors, pending hyperparameter tuning.
- TBD-4: Hosting platform for GPU-backed offline training (local workstation vs. Google Colab vs. cloud GPU instance).
- TBD-5: Final UI design and explanation-panel layout, pending wireframe approval.

C.2 Future Enhancements

- Incorporate a Generative Adversarial Network (GAN)-based negative sampler (as in IRGAN) to improve ranking quality.
- Extend the attention-fusion layer to a co-attention mechanism for joint reasoning over reviews and images.
- Introduce Deep Reinforcement Learning for real-time, reward-driven re-ranking of recommendation slates.
- Add cross-domain recommendation support (e.g., transferring knowledge between a media catalogue and an e-commerce catalogue).
- Adopt knowledge distillation to compress the model suite for low-latency, edge/mobile inference.